

CONVERGE • THE SPINE • AUTONOMOUS SYSTEMS

Converge

Compile intent into **autonomous systems** —
a fixed sequence of **seven passes**, each ending at a gate.

An AI-native engineer does not write the system. They lower a fuzzy requirement through a repeatable descent until a machine can execute it without supervision.

THE END-TO-END PROCESS

`cvg-aut-systems-spine`

— THE THESIS

◆ WHAT CONVERGE IS

You don't write the system. You compile intent into it.

Each pass drops one level of altitude, binds the right engine for that altitude, and ends at a gate that must pass before the next begins. The discipline is the gate. Today the gate is you — you confirm, you approve, you review. The dark factory replaces each human gate with a runnable eval. The passes do not change; only who stands at the gate.

EVERY PASS

Lowers altitude

Intent → structure → plan → tasks → code. Each step is closer to something a machine runs.

BINDS

An engine

The right tool for the altitude — Chat to think, Code to build, Codex to attack, Kimi to crank.

ENDS AT

A gate

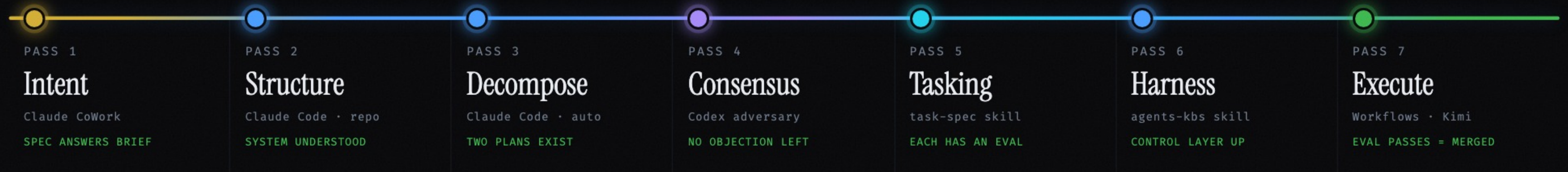
Converged means an eval passed — not that you feel done. The gate is the unit of trust.

— THE SPINE

◆ SEVEN PASSES • ONE FORK • ONE GATE

The whole method, on one line.

Intent → Structure → Decomposition → Consensus → Tasking → Harness → Execution. Each output is the next input. The chain descends from a fuzzy brief to merged code.



THE INVARIANT

Every pass lowers altitude, binds an engine, ends at a gate. **You are converged when the eval passes – not when you feel done.**

◆ INTENT • RECEIVE & COMPREHEND THE BRIEF

The client hands you the **problem**. You produce the **spec**.

ENGINE • CLAUDE COWORK (PROJECT)

IN • BRD-ANALYTICAL-BACKBONE.PDF

OUT • TECH-SPEC-ANALYTICAL-ENGINE.PDF

UNDERSTAND

Read

the BRD like a senior engineer – real problem, who hurts, their numbers



INTERROGATE

Grill

the 2-3 questions that most change how we build it



CRYSTALLIZE

Spec

the technical answer to their problem – your consulting deliverable

GATE

You can state the client's problem back correctly, and the **tech-spec answers it**. Born as a PDF – a consensus object you circulate before any code. No premature implementation the brief didn't justify.

— PASS 2 OF 7 • ALTITUDE: SYSTEM

◆ STRUCTURE • COMPREHEND THE SPEC AGAINST THE REPO

Open the **brownfield**. Understand a system you didn't write.

ENGINE • CLAUDE CODE • ON THE REPO

IN • TECH-SPEC + SRC/

OUT • SESSION UNDERSTANDING (NO FILE)

COMPREHEND

Explain

the system end to end – the brownfield + what the spec adds



GROUND

Check

does the spec match what's actually in src/ – schema, generator, raw.*?



CONFIRM

Map

a clean component + dependency map – what to build, in what order

GATE • NO ARTIFACT

You can explain the full system, consistent with the spec and the real repo. **Pass 2 produces no file** – its output is the loaded understanding in the session. Keep it open into Pass 3: the understanding is the handoff.

— PASS 3 OF 7 • ALTITUDE: PLAN

◆ DECOMPOSITION • BREAK IT INTO PLANS

Two plans, along the **natural** seam.

ENGINE • CLAUDE CODE (AUTO) • SAME SESSION

OUT • SKETCH/DUCKDB-DBT-MED-ARCH.PLAN

OUT • SKETCH/FAST-API-MCP.PLAN

DECOMPOSE

Seam

split the work – transformation vs serving – and justify the boundary



PLAN A

Medallion

dbt bronze → silver → gold over raw.*; deps, build order



PLAN B

Serving

FastAPI + MCP over the gold tables; the interface it consumes

GATE

Two plans exist along the spec's own seam; each lists features, dependencies, and a sane build order. **Plan altitude only** – no atomic tasks, no code yet. The session carries from Pass 2 – the understanding becomes the plan without leaving the session.

— PASS 4 OF 7 • ALTITUDE: PLAN (HARDENED)

◆ CONSENSUS • A DIFFERENT MODEL ATTACKS

Bring a different engine to refute the plan.

ENGINE • CODEX (ADVERSARY) ↔ CLAUDE (REVISES)

IN • BOTH PLANS + THE TECH-SPEC

OUT • PLANS SHARPENED IN PLACE

ATTACK

Refute

Codex, as a skeptic who didn't write it — default to refuted



GROUND

Drift?

check each plan against the tech-spec — contradictions, gaps



SHARPEN

Fix/Own

every objection FIXED in a plan or ACCEPTED with an owner

GATE • & THE FORK IS DECIDED

No open objection survives — nothing silently dropped. Cross-model disagreement is the point: Claude won't refute itself hard enough. **At the end of Consensus, the fork is decided** — does the trust boundary wrap the whole system, or each unit?

— THE FORK · DECIDED AT THE END OF CONSENSUS

◆ WHERE DOES THE TRUST BOUNDARY SIT?

One coupled system, or many independent units?

FORK A · PLAN-DRIVEN

One coherent spec

- Speckit / OpenSpec / AgentSpec
- One tightly-coupled system — pieces only make sense together
- Trust boundary wraps the whole system
- Verify: one end-to-end test

VS

FORK B · TASK-DRIVEN

Many atomic tasks

- task-spec + engines
- Loosely-coupled units — each stands alone (dbt models, MCP tools)
- Trust boundary wraps each unit
- Verify: per-unit runnable evals

BOTH FORKS RECONVERGE ON ONE GATE

eval passes = merged. The cut differs; the gate is identical. That shared gate is what makes Converge one method with two paths — and what lets either go dark-factory.

— PASS 5 OF 7 • ALTITUDE: ATOMIC UNIT

◆ TASKING • CUT THE WORK INTO EVAL-BACKED UNITS

Atomic. Self-contained. Engine-agnostic.

ENGINE • TASK-SPEC SKILL

IN • THE TWO HARDENED PLANS

OUT • TASKS/* PER PLAN, EACH WITH AN EVAL

CUT

Atomize

one indivisible unit per task – "build silver_orders", "the MCP query tool"



DESCRIBE

Tech, not agent

name the work + paths, not the harness agent – that comes next



BIND EVAL

Done = ?

a runnable eval defines done – dbt test passes, gold query returns N

GATE

Every task carries a runnable eval – no eval, not a task yet. Task-spec is **self-contained** (its own architect + scripts) and **vendor-portable** (Claude, Codex, Kimi, or manual). The spec is engine-agnostic; the eval is engine-universal.

◆ HARNESS · GENERATE THE BUILD-SYSTEM THE TASKS NEED

Build the rails the tasks will ride on.

ENGINE · AGENTS-KBS-TECH-STACK SKILL

IN · THE TASKS (THEY NAME WHAT THEY NEED)

OUT · .CLAUDE/ – AGENTS · KBS · RULES · CLAUDE.MD

READ NEEDS

Scope

the tasks touch dbt + duckdb + mcp – scaffold exactly those



SCAFFOLD

Agents + KBs

paired architect + developer per tech, grounded in real docs



EMIT

Cross-tool

AGENTS.md · Cursor · Copilot – every engine inherits one contract

GATE · ORDER NOTE

The control layer stands; agents are grounded. **Tasking before Harness on purpose** – the tasks are the requirements; the harness is fitted to exactly what they need, not built speculatively. The harness is the one artifact that outlives the project.

— PASS 7 OF 7 • ALTITUDE: CODE

◆ EXECUTION • BUILD UNTIL THE EVALS GO GREEN

The loop closes when the **eval** passes.

ENGINE • WORKFLOWS • AGENTSPEC • KIMI CRANK

IN • TASKS + GROUNDED HARNESS

OUT • MERGED CODE (DBT + MCP ON RAW.*)

DISPATCH

Right engine

Kimi cranks the mechanical models · Claude judges · Codex reviews



RUN EVAL

Green?

each task's eval runs – dbt test, gold query, the MCP responds



LOOP / MERGE

Until green

re-run until the eval passes – then merge

THE GATE THAT CLOSES THE LOOP

eval passes = merged. Each engine in its role; each task looped until its eval is green. Automate this gate and the human steps out – that is the dark factory.

— CONVERGE · THE INVARIANT

*“Every pass lowers altitude, binds an engine, ends at a gate. You are converged when the eval passes — **not when you feel done.**”*

REMOVE YOURSELF ONE GATE AT A TIME · WHAT'S LEFT IS THE DARK FACTORY